Šepet: sporočanje s šifriranjem od konca do konca kot oblačna mikrostoritvena aplikacija

Tibor Koderman, Amar Mehić, Eva Vidic
Projektna skupina 13 »Ključarji« · Fakulteta za računalništvo in informatiko, Univerza v Ljubljani
Prijava teme projekta pri predmetu Računalniške storitve v oblaku 2026/27
Repozitoriji: https://gitlab.koderverse.com/fri-mag/rso

***Povzetek*—Pri večini aplikacij za klepet moramo ponudniku zaupati, da sporočil ne bo bral ali jih predal tretjim; Telegram na primer klepetov privzeto ne šifrira od konca do konca [1], zahteve po dostopu oblasti do zasebnih sporočil pa so vse pogostejše. Razvili bomo aplikacijo Šepet za sporočanje ena na ena in v skupinah z oblikovanim besedilom in slikami. Vsebina se šifrira na napravi pošiljatelja s standardnim protokolom MLS [2] in dešifrira šele na napravah prejemnikov, zato strežnik hrani in posreduje le šifrirane podatke. Sistem bomo zgradili iz sedmih mikrostoritev in ga namestili v oblak Azure (Kubernetes).**

***Ključne besede*—šifriranje od konca do konca, MLS, mikrostoritve, Kubernetes, Azure**

I. Varnostni model

Strežnik je zanesljiv za dostavo, ne pa za zaupnost. Iz tega izhajajo štiri odločitve:

- **Vse v pogovoru je šifrirano**: sporočila, slike, imena skupin, pa tudi tipkanje, potrdila o branju in reakcije, ki potujejo kot šifrirana sporočila MLS. Strežnik vidi le, kdo je v katerem pogovoru.
- **Pregleden dnevnik ključev**: javni ključi naprav so v dnevniku z Merklovim drevesom, zato strežnik zamenjave ključa ne more prikriti; uporabnika lahko primerjata tudi varnostno številko s kodo QR.
- **Prijava brez gesel**: s ključi passkey (WebAuthn), odpornimi proti ribarjenju.
- **Obvestila brez vsebine**: potisno obvestilo napravo le zbudi, aplikacija sporočilo prenese, dešifrira in prikaže sama.

II. Tehnologije in razvojno okolje

Tabela I povzema izbiro. Storitve pišemo v .NET 10 po načelih Clean Architecture, vsaka v svojem repozitoriju z lastnim Helm chartom in cevovodom GitLab CI.

III. Arhitektura

Shema je na sliki 1. Aplikacija dostopa do storitev prek prehoda API, ki preverja žetone JWT in usmerja zahteve (REST, GraphQL, WebSocket). Kadar storitev potrebuje takojšen odgovor, kliče drugo storitev prek gRPC (npr. preverjanje članstva pri pošiljanju), dejstva, ki zanimajo več storitev, pa objavi kot dogodek v NATS JetStream. Vsaka storitev s trajnimi podatki ima svojo bazo, šifrirane slike pa potujejo neposredno v Blob Storage.

Tabela I
Izbrane tehnologije

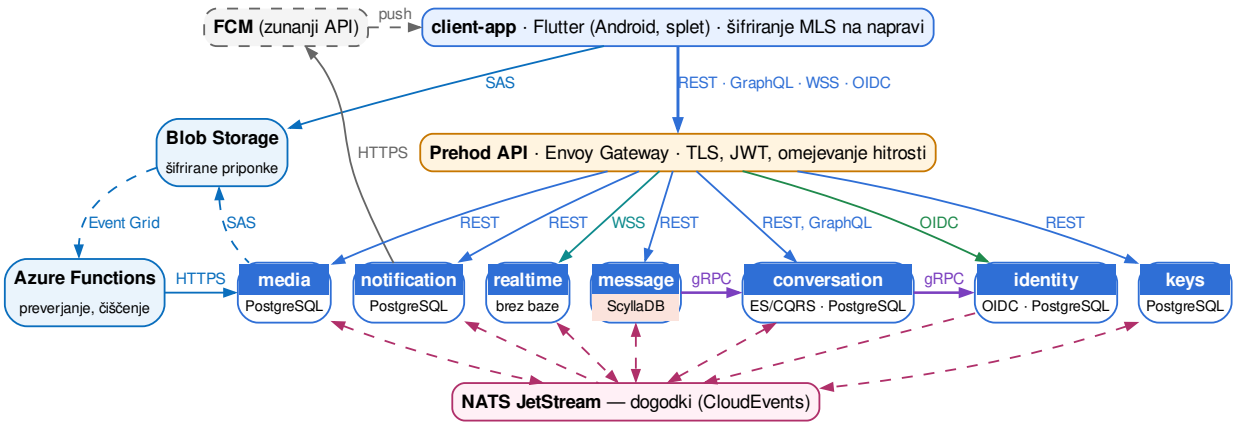
Področje	Izbira
Odjemalec	Flutter (Android in splet)
Šifriranje	MLS (OpenMLS); rezerva Signal Protocol
Storitve	.NET 10, ASP.NET Core, Clean Architecture
Baze	PostgreSQL, ScyllaDB, Azure Blob Storage
Komunikacija	REST, GraphQL, gRPC, WebSocket, NATS JetStream
Identiteta	lasten ponudnik OIDC v .NET (OpenIddict, passkeys)
Platforma	Docker, Kubernetes (AKS), Helm, Envoy Gateway, Terraform
CI/CD	GitLab CI
Opazovanje	Prometheus, Grafana, Loki
Ostalo	Azure Functions, Firebase Cloud Messaging
Orodja	Rider, VS Code, Android Studio, Docker Compose, kind

Tabela II
Mikrostoritve in njihove funkcionalnosti

Storitev	Funkcionalnosti
identity	ponudnik OIDC: prijava s passkey, organizacije, vloge, naprave
keys	javni ključi naprav, paketi ključev MLS, pregleden dnevnik ključev
conversation	pogovori in skupine, člani, prošnje za pogovor, časovnik izginjanja; Event Sourcing, CQRS, GraphQL
message	hramba šifriranih sporočil s samodejnim brisanjem (TTL v ScyllaDB), vrstni red sprememb skupine po epohi MLS, sinhronizacija naprav
realtime	WebSocket: dostava v živo, prisotnost, šifrirani efemerni dogodki
media	nalaganje in prenos šifriranih slik, omejitve velikosti
notification	potisna obvestila brez vsebine (Firebase Cloud Messaging)

IV. Mikrostoritve

Tabela II našteva sedem mikrostoritev. Message uporablja ScyllaDB, realtime je brez baze, ostale imajo vsaka svojo bazo PostgreSQL. Dopolnjujeta jih aplikacija v Flutterju in funkcija Azure Functions, ki preveri naložene slike in briše potekle. Delo smo razdelili tako: Tibor Koderman (identity, keys, platforma), Amar Mehić (conversation, message, realtime), Eva Vidic (media, notification, funkcije, aplikacija).



Slika 1. Začetna shema arhitekture. Modro: REST in GraphQL prek prehoda API; zeleno: OpenID Connect; vijolično: gRPC; rožnato črtkano: dogodki NATS JetStream; svetlo modro: storitve Azure.

V. Primeri uporabe

A. Seznam primerov uporabe

- registracija in prijava s ključem passkey;
- iskanje uporabnika in prošnja za pogovor, ki jo prejemnik sprejme ali zavrne;
- pošiljanje sporočil z oblikovanim besedilom in stanjem dostave (poslano, dostavljeno, prebrano);
- pošiljanje slik;
- skupine: ustvarjanje, dodajanje in odstranjevanje članov;
- obvestila, ko aplikacija ni odprta;
- preverjanje varnostne številke s kodo QR;
- izginjajoča sporočila: časovnik nastavi pogovor, naprave in strežnik pa sporočila in slike po izteku izbrišejo;
- upravljanje organizacije in vlog (skrbnik).

B. Kompleksnejši primer: slika v skupinskem pogovoru

1) Aplikacija sliko šifrira z naključnim ključem in pri storitvi media dobi povezavo za nalaganje.
2) Slika se naloži v Blob Storage; funkcija Azure jo preveri in obvesti media.
3) Aplikacija pošlje šifrirano sporočilo s ključem slike storitvi message, ta prek gRPC pri conversation preveri članstvo.
4) Message sporočilo shrani v ScyllaDB in objavi dogodek v NATS.
5) Realtime ga dostavi povezanim članom, notification pa ostalim pošlje potisno obvestilo.
6) Prejemniki sporočilo in sliko dešifrirajo na svojih napravah.

Literatura

[1] Telegram, »FAQ: Secret chats«, https://telegram.org/faq.
[2] R. Barnes et al., »The Messaging Layer Security (MLS) Protocol«, RFC 9420, IETF, 2023.